

问题:

1.你之前有过多人协同开发的经历吗? 如果有, 你们是使用什么方式分工协作的?

A: 没有

2.思考一下, Git 为什么要设计 “暂存-提交” 两个步骤?

A:灵活精细地控制每次提交包含哪些修改,方便版本更新

3.git branch 和 git branch -a 的区别是什么? 查阅资料并回答。

A:

git branch

不带参数, **只列出本地分支**, 看不到远程仓库的分支。前面带 * 号的代表当前正在使用的本地分支。

git branch -a

-a 是 --all 的缩写, **列出所有分支: 本地分支 + 远程跟踪分支**。

远程分支一般会显示为 origin/main 这种格式。

为什么要学习 Git?

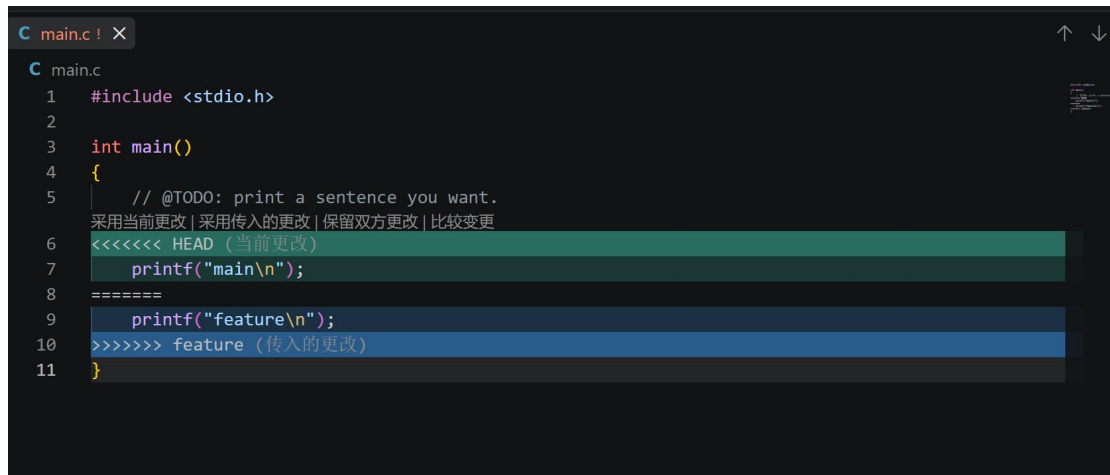
而 Git 是一种版本管理系统，Git 操作是基于分支的，当下环境衍生出多种优秀的分支管理策略，可以保证不同分支各司其职，避免多人协作过程中代码冲突、代码版本出现问题。

第一个文件主要讲的是 Commit message 和 Change log 的意义与写法规范，Git 每次提交代码，都要写 Commit message（提交说明），说明本次提交的目的，否则就不允许提交，可以直接从 commit 生成 Change log，用来说明与上一个版本差异的文档。Commit message 和 Change log 都有严格的规范，在多人协作开发的情境下，可以清楚地说明每一次改动的内容，版本变化的情况。

从第二个文件主要讲的是 Gitflow 的使用规范和 workflows，Gitflow 是一种 Git 工作流程，是一种 Git 分支管理策略。有助于团队开发过程不同分支各司其职。

截图：

```
    printf("Hello\n");
}
yimo17505582007@3:~/lgy$git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
yimo17505582007@3:~/lgy$ |
```



```
C main.c ! X
C main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
        采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
6      <<<<<< HEAD (当前更改)
7      printf("main\n");
8      =====
9      printf("feature\n");
10     >>>>>> feature (传入的更改)
11 }
```

实验步骤:

1. 下载配置环境。
2. 在 github 上使用初始模板仓库建立自己的仓库在 main.c 文件中改一下输出的内容并进行一次 commit。
3. 阅读 Commit Message 规范和 Git Flow 分支控制
4. 学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交。

随后将 feature 分支 merge 到 main 分支（即切换回 main 分支执行 git merge feature），并处理发生的合并冲突，即回到 vscode, 把它改成在 main 中运行的模样（第二张图片），然后提交。